

Finding optimal solutions to the Fifteen Puzzle

François Pottier

Christian Rinderknecht
rinderkn@caesar.elte.hu

19th August 2026

1 Preliminaries

1.1 The Fifteen Puzzle

The Fifteen Puzzle consists in a square board, divided into sixteen squares (or *positions*), upon which are placed fifteen sliding square tiles numbered from 1 to 15, leaving one position empty, or *hole*. From a given configuration, the goal is to slide the tiles one by one and reach another given configuration. One valid *move* consists in sliding one tile adjacent to the hole, which is equivalent to exchanging their respective positions. (For a few months, this game was extremely popular in the USA and Europe during the year 1880.)

Abstractly, the space of the configurations of the Fifteen Puzzle is a graph where each vertex represents a configuration and its successors are the configurations resulting from one valid move. Therefore, each vertex has two, three or four successors. The purpose of the game then consists in finding a path through the graph joining a given vertex and an end-game vertex. In the present project, the start configuration will be variable and the final configuration will be that found at the left in Figure 1. It is possible that the initial and final configurations are not connected, in particular, the variant where the final configuration is altered so that the tiles 14 and 15

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

10	6	4	12
1	14	3	7
5	15	11	13
8		2	9

Figure 1: Final configuration and a random one

are swapped. This fact was already proved in 1879 and a simpler proof was given by Archer (1999).

1.2 Depth-first search with iterative deepening

The algorithmic difficulty resides in the huge size of the graph. The number of possible configurations, that is to say, the number of vertices of the graph, is $16!$, about $2 \cdot 10^{13}$. It is mathematically proven that the graph is divided into two *connected components*, whose number of vertices is thus of the magnitude 10^{13} each. Clearly, such a graph cannot be stored in the memory of an up-to-date computer, therefore an algorithm for finding the shortest paths (Dijkstra, 1959, Cormen et al., 2009, Sedgewick and Schidlowsky, 2003) is useless (its memory usage is linear with respect to the number of vertices in the graph). Classical algorithms for depth-first or breadth-first traversals are also useless: indeed, in order to guarantee termination, these algorithms colour the visited vertices, yielding a memory consumption proportional to the number of vertices in the worst case.

Consequently, we must trade some efficiency for a reduced memory usage. How can we proceed?

An original answer to that question was given by several authors, amongst them Korf (1985). On the one hand, because breadth-first traversals, like Dijkstra algorithm and A* (best-fit), rely on queues whose size may be linear in the number of vertices, they are useless. On the other hand, depth-first traversals use a stack whose size is only that of the path being currently explored. Nevertheless, let us keep in mind that it is impossible to associate a colour to each vertex, for lack of memory, therefore we must resort to variants which do not colour the vertices. But, of course, the lack of colouring will impede the detection of cycles... In order to ensure the termination of the program, we must stop the exploration at a given depth k . Since we do not know in advance the length of the optimal path we seek, we have to perform several depth-first traversals, with increasing depths k . This strategy is called *depth-first search with iterative deepening*. If the threshold k is incremented by 1 at each iteration, we are certain that the first path reaching the goal (that is, the final configuration) is optimal. This approach may seem too slow, because we traverse several times the vertices close to the initial vertex, but the bulk of the work is done far away from the initial vertex, so this repetition is not overly slow, as shown by Korf (1985).

1.3 Using a lower bound on the distance

An independent and equally interesting idea lies at the heart of the A* algorithm (Hart et al., 1968, 1972), which is an improvement upon Dijkstra's algorithm, although they are both linear in space. The point is to use an external notion of distance, for example, the geometric distance between ver-

tices, in order to deduce a lower bound on the topological distance on the graph. That approximation of the distance is used to explore in priority paths which seem to approach the goal. This strategy, known as *best-fit search*, allows the exploration of a path to stop as soon as it is known that it cannot be extended into an optimal path, a process known as *pruning the search tree*.

For instance, let us imagine a road map of Hungary, represented as a graph whose vertices are towns and cities, and the edges are roads. The edges are annotated with the distance of the corresponding road, in kilometres. The straight line distance provides then a lower bound on the road distance. Moreover, it is easy to compute if we know the geographic coordinates of each town, so it can be used to guide the search towards shorter paths. Let us assume, for instance, that we seek an optimal path from Budapest to Debrecen. The road distance between Budapest and Debrecen is lower than the sum of the road distance between Budapest and Győr, on the one hand, and the straight-line distance between Győr and Debrecen, on the other hand. Thanks to this knowledge, the A* algorithm would not consider the paths from Budapest to Győr and would explore more promising paths, from which the optimal path will be found. This reckoning seems obvious to a human, but a simpler algorithm, like Dijkstra's algorithm, would examine all paths in larger and larger concentric circles around Budapest, including roads to Győr and Vienna, before discovering the optimal path to Debrecen.

Can this idea be applied to the Fifteen Puzzle? In other words, do we have a simple way to determine a lower bound on the distance between two configurations? The answer is yes. Since each move is a slide of a tile next to the hole, the number of moves to bring a given tile from its current position to its final position is at least the distance between these two positions. This minimal distance is the *Manhattan distance* and is easy to compute. Moreover, since each move affects exactly one tile, this reasoning holds independently for all tiles and the number of moves to reach the final configuration is at least the sum, for each tile, of the minimal distances between the current configuration and the final one. This is the first idea to reduce the space search: use an external minimal distance.

1.4 Combination

The two ideas presented above can be combined to define depth-first searches with iterative deepening, relying on a lower bound of the distance. The simplest of these algorithms is IDA* (Korf, 1985), where the threshold k can be incremented beyond the value 1 between two iterations: the value of k is the estimation of the length of an optimal path passing through the closest discovered vertex yet to be visited (because it is beyond the threshold) during the previous iteration. A rather readable pseudo-code for IDA* is given by Reinefeld and Marsland (1994), in figure 1.

1.5 Refinements

Several improvements are possible, based on the ideas described above.

1.5.1 Incremental computation of the estimated distance

The lower bound on the distance between a given configuration and the final configuration can easily be kept up-to-date when the former is modified by a given move. This incremental calculation saves times.

1.5.2 Improvement of the distance approximation

In the case of the Fifteen Puzzle, the Manhattan distance can be improved upon by taking into account the *linear conflicts*. The idea is that if two tiles are on the right row or column, but their respective positions are reversed, then one of them will have to leave temporarily that row or column. The Manhattan distance does not use this fact, and we can therefore add 2 without risking to overestimate the real distance. In certain positions, there are several such conflicts, and the compounded improvements yield an approximation much better than the Manhattan distance (Hansson et al., 1992). On the one hand, the exploration of a vertex takes more time because the calculation of the distance becomes more complex; on the other hand, the number of vertices to examine greatly decreases, and this results in a overall speed-up.

1.5.3 Pruning series of redundant moves

An important improvement consists in immediately pruning series of redundant moves, which are defined as follows. In the Fifteen Puzzle, each move is reversible: it can be followed by a symmetric move, so their composition leaves the board invariant. Clearly, any path containing two of these moves cannot be optimal, and can be ignored, or *pruned*. If the moves are tagged by the letters U (up), D (down), L (left) and R (right), then any path containing the words UD, DU, LR or RL can be pruned. This test can be implemented by presenting the tag of each move to a finite automaton, which may trigger a pruning when certain states are reached. This simple optimisation yields a considerable speed-up. Indeed, the branching factor, that is, the number of successors for each vertex, is effectively reduced from 4 (at worst) to 3 (at worst), which means that the time complexity decreases from $O(4^k)$ to $O(3^k)$.

In order to recognise the four preceding words, an automaton with five states is enough and it can be constructed by hand. But these words are not the only one being redundant. For example, we can check that, starting from any configuration, the series RDLURD and DRULDR are either both impossible or lead to the same configuration. In other words, these series

of moves identify cycles of length 12 in the graph of configurations. Consequently, if there exists an optimal path containing the sequence RDLURD, then there exists also one which does not contain it and, instead, includes the sequence DRULDR. Of course, the inverse choice is also acceptable. Following this reasoning, we can forbid four words of length 12, beyond the four words of length 2 already identified, and we can recognise these words with an automaton containing 29 states. Pushing further this method, we can identify longer and longer series of redundant moves. Naturally, it is not feasible to discover these sequences and construct by hand the corresponding automaton, but it is possible to apply machine learning (Taylor and Korf, 1993).

2 Subject and guidelines

The goal is to find an optimal path between an arbitrary configuration and the final configuration at the left in Figure 1, and doing so within a reasonable time frame using **Java** as the programming language, without relying on specialised libraries (only the standard library is acceptable).

The time complexity is expected to be *exponential* with respect to the length of the optimal path. Depending on the initial configuration, a solution can be 10 times or even 10.000 times slower than others. Consequently, it is important that any optimisation has an impact other than “simply” gaining 50%.

The program will read the initial configuration from the standard input. A configuration is written textually as a list of sixteen integers ranging from 0 to 15, separated by blanks and ended by a return (end-of-line control character). The i th integer in the list denotes the number of the tile located at the i th square on the board, the tile number 0 being the hole. The squares are numbered from 0 to 15, rightwards and top-down, so the final configuration at the left in Figure 1 is written

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

and the configuration written as

```
10 6 4 12 1 14 3 7 5 15 11 13 8 0 2 9
```

is the one at the right in Figure 1.

The output of the program is sent to the standard output and consists in one line of summary, optionally followed by a path. The summary is either **FAILURE** if there are no paths between the initial and final configurations, or **OK n** if such paths exist and the shortest has length n . In the latter case, one of such paths is printed as a series of configurations (one per line).

A test suite is supplied, where each file whose name is matched by the regular expression `*.[0-9]+` contains an initial configuration, and the file

whose name is derived from it by adding the suffix `.ref` contains the summary line which must be produced by your program. (The optimal paths are not unique, therefore none is given to avoid confusion.) These tests are of very different level of difficulty. As far as the tests proposed by Korf (1985), in table 1, are concerned, an approximation of the difficulty is given by the column “Total Nodes”. The number of vertices explored by your program will not necessarily equal the number of vertices explored by Korf’s algorithm, because the optimisations are likely to differ, but that table clearly shows that the difference in difficulty between two tests can be of several orders of magnitude.

Here is a proposed plan of action. The order of the different phases is only indicative and you can proceed differently.

1. Determine what operations must be supported by the data structure which implements the graph of configurations. It should be a graph
 - where a vertex (the final configuration) is distinguished;
 - where the edges are tagged by the letters U, D, L or R;
 - where, given a vertex, we can get the list of the outgoing edges;
 - where, given a vertex, we can query a lower bound on its distance to the final configuration.

This specification should be implemented in **Java** by an interface named **GraphConf**.

2. Write a class implementing the interface **GraphConf**. Beyond the operations above, you should add the transformation of a configuration given as a line of text into a vertex in the graph of configurations. Given the overall expected complexity of the program, it is not useful to worry too much about the efficiency of these operations, at least at first, and it is best to focus instead on simplicity. (Later on, you could improve them by using precomputed tables to speed up the calculation of the estimated distance.)
3. Write an implementation of the IDA* algorithm. The algorithm initiates a search from a given vertex and produces an optimal path reaching the final vertex, or fails. For guidance, you may consult the definition of the A* algorithm by Hart et al. (1968), who explain the principle of the estimation of the distance, as well as the original presentation of the IDA* algorithm by Korf (1985). A readable pseudo-code for IDA* is given by Reinefeld and Marsland (1994). This algorithm is very simple and its implementation should only spread over a hundred lines. Do not hesitate to insert printing instructions to display in real time the number of iterations performed, the number of vertices explored at each iteration etc. (Use the error output, not the standard output, for this.)

4. Combine everything into a program satisfying the requirements. Check that it passes successfully the easiest test cases, for example, `easy.*`, `korf.009` and `korf.012`.

This is the minimal effort. To go beyond, you could pursue with the following additional steps.

1. You may generate your own initial configuration randomly. In that case, the paper by Archer (1999) is useful in order to avoid producing unreachable configurations. (The IDA* algorithm can only solve a problem if there is a solution. In the absence of a solution, it cannot check that all the space of configuration has been explored.) Some reflection is needed to understand how to determine whether a configuration is reachable or not.
2. Generalise your implementation of the IDA* algorithm so the tag of each move is submitted to an automaton and the exploration is abandoned immediately if the automaton recognises a series of moves. Write by hand an automaton recognising the sequences UD, DU, LR and RL. Combine everything and measure the speed-up.
3. Update incrementally (at each move) the approximation of the distance associated to the current vertex (§1.5.1).
4. Refine the estimation of distance so it takes into account the linear conflicts (§1.5.2). The exact definition of the new estimation is given by Hansson et al. (1992). Some care is needed to implement it efficiently, and the authors do not provide all the details.

References

- Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.
- Aaron F Archer. A modern treatment of the 15 puzzle. *American Mathematical Monthly*, 106:793–799, November 1999.
- Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, third edition, 2009.
- Edsger Wybe Dijkstra. A note on two problems in connection with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- Othar Hansson, Andrew Mayer, and Moti Yung. Criticizing solutions to relaxed models yields powerful admissible heuristics. *Information Sciences*, 63(3):207–227, September 1992.

- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, July 1968.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. Correction to “A formal basis for the heuristic determination of minimum cost paths”. *SIGART Newsletter*, 37:28–29, 1972.
- Haym Hirsh. Playing with AI. *IEEE Intelligent Systems*, pages 8–18, November 1999.
- Richard E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, September 1985.
- Richard E. Korf and Larry A. Taylor. Finding optimal solutions to the Twenty-Four Puzzle. In *National Conference on Artificial Intelligence (AAAI)*, pages 1202–1207, 1996.
- Alexander Reinefeld and T. A. Marsland. Enhanced iterative-deepening search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(7):701–710, 1994.
- Robert Sedgewick and Michael Schidlowsky. *Algorithms in Java: Graph Algorithms*. Addison-Wesley, 2003.
- Larry A. Taylor and Richard E. Korf. Pruning duplicate nodes in depth-first search. In *National Conference on Artificial Intelligence (AAAI)*, pages 756–761, 1993.